CSED233 · 데이터구조 · 포스텍

Ch4. List, Stack, Queue — 선형 자료구조

1. List (리스트)

정의 & 개념

- **List(리스트)**: 원소의 중복을 허용하는, 유한하고 순서 있는 수열

$$L = \langle a_1, a_2, \ldots, a_n \rangle$$

- n = length (size), $n = 0$이면 empty list
- **Head** (front) ↔ **Tail** (rear)
- "current position": 원소 사이 fence로 표시
  - 예: $\langle 20, 23 | 12, 15 \rangle$ 에서 current에 10 insert → $\langle 20, 23, 10, 12, 15 \rangle$

Sorted vs Ordered (중요한 구분)

- **Ordered**: 원소 간 **순서(position)** 가 정의됨 — 예: $\langle 20, 23, 12, 15 \rangle$ (위치 있음, 크기 무관)
- **Sorted**: key 값 기준 정렬 — 예: $\langle 12, 15, 20, 23 \rangle$
- → **모든 List는 ordered지만, sorted는 아닐 수 있다.**

표준 Operations

Op	의미
Insert(x, p, L)	위치 p에 x 삽입
Delete(p, L)	위치 p 원소 삭제
Next(p, L)	p 다음 위치 포인터
Previous(p, L)	p 이전 위치 포인터

Locate(x, L)	x의 위치 찾기
Retrieve(p, L)	p에서 원소 값
MakeNull(L)	empty로 만들기
First(L)	첫 위치
END(L)	마지막 다음(past-end)

2. Array-Based Implementation

구조

```
char L[max];
int last = 현재 원소 개수
```

Array-Based List의 위치 규칙: 리스트의 논리적 위치 p는 1-based이며, C 배열의 실제 인덱스는 p−1이다. last는 마지막 원소의 인덱스가 아니라 현재 저장된 원소의 개수 n을 의미한다. Insert(x,2,L)은 두 번째 논리적 위치에 x를 삽입한다. 초기 배열 [a][b][c][d][_][_]에서 실제 인덱스 1~3의 원소 b,c,d를 오른쪽으로 이동시키고 인덱스 1에 x를 저장하므로 결과는 [a][x][b][c][d][_]가 된다. 원소가 n개일 때 유효한 삽입 위치는 1≤p≤n+1이며, 삽입으로 이동하는 원소 수는 n−p+1개다. 배열 용량이 충분하다는 전제에서 최악의 삽입 시간복잡도는 O(n)이다.

Insert(x, 2, L) 동작

```
초기: [a][b][c][d][_][_]    last=4
삽입: [a][x][b][c][d][_]    last=5
```

- p=2에 x 넣으려면 **p..last 전부 오른쪽으로 한 칸 shift** 필요
- **Running time =** $O(n)$ (linear, worst case 모든 원소 이동)
- Delete, Locate도 마찬가지 → $O(n)$

3. Pointer-Based (Linked List(연결 리스트))

Singly-Linked List(단일 연결 리스트)

```
struct cell {
  char info;
  cell* link;
};
```

- **Node = info + link**
- **L → [a] → [b] → [c] → [d] → NULL**

Fence(펜스 · 현재 위치 표시) (current position) 표현

Option 1: `p`가 현재 원소 **직접** 가리킴

- 문제: Insert 시 선행 노드의 link를 바꿔야 하는데 직접 접근 불가 → **선행 노드 access 불편**

Option 2 (One-step-ahead convention): `p`가 **선행 원소**를 가리킴

- `p`의 정의가 달라짐: `p`는 "b" 노드 → current는 "c" 의미
- → Insert 쉬움

Empty list / 왼쪽 partition 빈 경우 처리

- **Solution 1**: if로 special case
- **Solution 2 (권장)**: **Dummy header(더미 헤더) node** 도입
  - `L → [dummy] → [a] → [b] → ...`
  - 모든 경우 일관 처리 → 코드 단순화

ADT 위치와 구현 포인터의 구분

ADT 위치와 구현 포인터의 구분: 추상적 List 연산에서 p는 1-based 논리적 위치를 의미한다. Insert(x,p,L)은 위치 p의 원소 앞에 x를 삽입하고, Delete(p,L)은 위치 p의 원소를 제거한다. Singly-Linked List에서 이를 구현할 때는 선행 노드를 가리키는 포인터 q를 별도로 사용한다. q->link가 목표 원소를 가리킬 때, 삽입은 새 노드를 q와 q->link 사이에 연결하고, 삭제는 q->link를 리스트에서 분리한다. q가 이미 주어졌다면 두 연산은 O(1)이다. 하지만 논리적 위치 p만 주어졌고 해당 위치를 처음부터 탐색해야 한다면 전체 연산은 최악 O(n)이다. Dummy header를 두면 첫 번째 원소에 대해서도 선행 노드가 존재한다.

Q1. Insert(x, p, L)의 시간복잡도는?

> 힌트

연결 리스트에서 위치 p의 포인터가 이미 주어지면 탐색이 필요 없다.

> 정답

개념은 힌트 참고.

포인터 조작만 하면 되므로 $O(1)$ — 탐색 비용 없음.

4. Doubly-Linked List(이중 연결 리스트)

구조

```
struct cell {
  char info;
  cell* prev;
  cell* next;
};
```

- 양방향 탐색 가능, 선행 노드 접근 $O(1)$

5. Cursor-Based (Simulated Pointer(포인터))

개념

- **배열 index로 pointer simulate**
- 한 배열에 여러 list 저장 가능
- Link = 0 → list tail

예시: 두 리스트 저장

index	info	link
1	c	0
2	a	4
3	y	0
4	b	1

```
5  -  -
6  x  3
```

- Head of L_1 = 2 (a → b → c → 0)
- Head of L_2 = 6 (x → y → 0)

Cursor-Based List의 시간복잡도: Cursor-based list는 배열 인덱스를 포인터처럼 사용하고, 각 원소의 link 필드에 다음 원소의 인덱스를 저장한다. link=0은 다음 원소가 없음을 뜻한다. 삽입·삭제 위치의 선행 cursor가 이미 주어져 있고, 사용 가능한 슬롯을 free list에서 O(1)에 할당·반납할 수 있다면 Insert/Delete는 O(1)이다. 선행 cursor를 찾는 탐색이나 빈 슬롯을 찾는 선형 스캔이 필요하면 최악 O(n)이다. 특정 cursor 인덱스를 직접 이용한 Retrieve는 O(1)이지만, k번째 원소를 Head부터 순차적으로 찾는 Retrieve는 O(n)이다.

6. Stack(스택) (LIFO(후입선출))

정의

- **Last-In, First-Out**: 모든 insert/delete가 **한 끝 (Top)** 에서 발생

  $$S = \langle a_1, a_2, \ldots, a_n \rangle,\ \mathrm{Top} = a_1$$

- 어떤 list 구현(pointer/array/cursor)으로도 구현 가능

Operations

- Push(x, S): $S = \langle x, a_1, \ldots, a_n \rangle$
- Pop(S): $S = \langle a_2, \ldots, a_n \rangle$
- Top(S): 첫 원소 반환
- MakeNull(S), IsEmpty(S)

Stack(스택) 구현

Array-based:

- Top 위치를 *I*로 잡으면 (*I*은 배열의 끝 위치, 즉 last가 가리키는 쪽 끝을 의미): push/pop $O(1)$
- Top을 고정 위치 1로 잡으면: push마다 shift → $O(n)$
- 한 배열에 **2개 stack 저장** 가능 (양끝에서 서로 마주보고 grow)

Linked Stack: 단일 연결 리스트의 head를 top으로 사용하면 Push와 Pop을 포인터 조작으로 O(1)에 수행할 수 있다. Stack의 활용 예: 함수 호출과 복귀를 관리하는 call stack이 대표적이다. 다만 call stack은 반드시 연결 리스트로 구현되는 것이 아니며, 일반적으로 연속된 스택 메모리와 스택 포인터를 이용해 관리할 수도 있다.

7. Queue(큐) (FIFO(선입선출))

정의

- **First-In First-Out**
- **Front** (삭제 위치), **Rear** (삽입 위치)

Operations

- Enqueue(x, Q): rear에 x 추가
- Dequeue(Q): front 원소 제거
- Front(Q): front 원소 반환
- MakeNull(Q), IsEmpty(Q)

Stack 및 Queue 연산의 사전 조건: Stack의 Push(x,S)는 top에 새 원소를 추가하며, Pop(S)는 top 원소를 제거하고, Top(S)는 제거하지 않고 반환한다. Pop과 Top은 스택이 비어 있지 않을 때만 정상 수행된다. 고정 크기 배열 스택에서 공간이 없을 때의 Push는 overflow를 처리해야 한다. Queue의 Enqueue(x,Q)는 rear에 원소를 추가하고, Dequeue(Q)는 front 원소를 제거하며, Front(Q)는 제거하지 않고 반환한다. Dequeue와 Front는 큐가 비어 있지 않을 때만 정상 수행된다. 고정 크기 배열 큐에서 공간이 없을 때의 Enqueue는 overflow를 처리해야 한다.

8. Array-Based Queue(큐): Drifting Problem

문제 상황

```
size 6 array, enqueue a,b,c → [a][b][c][_][_][_]   F=0 R=2
dequeue twice               → [_][_][c][_][_][_]   F=2 R=2
enqueue d,e,f               → [_][_][c][d][e][f]   F=2 R=5
```

- **뒤 공간 없음** → 앞은 비었는데 enqueue 불가
- 이것을 **"drifting queue"** 문제라고 함

해결: Circular Array (환형 배열)

Empty vs Full: 왜 추가 상태 정보가 필요한가? F가 첫 원소, R이 마지막 원소를 가리키는 원형 큐에서 (R+1)%size==F는 빈 상태와 가득 찬 상태 모두에서 나타날 수 있으므로, 두 인덱스만으로는 원소 수를 항상 결정할 수 없다. 해결 방법은 세 가지다. 1. cnt 유지: cnt==0이면 Empty, cnt==size이면 Full이다. 2. Empty/Full 상태를 판별할 추가 플래그 유지: 인덱스가 모호한 상태에서 플래그로 두 경우를 구분한다. 3. 슬롯 하나 예약: 최대 size−1개의 원소만 저장한다.

```
enqueue(x):
R = (R+1) % size
array[R] = x
```

Circular Queue의 인덱스 규칙과 Empty/Full 판정: 여기서는 F를 첫 번째 원소의 인덱스, R을 마지막 원소의 인덱스로 정의한다. 배열의 크기는 size이고, 유효한 인덱스는 0부터 size−1까지다. 초기 상태는 F=0, R=size−1, cnt=0이다. Enqueue는 R=(R+1)%size로 마지막 원소의 위치를 한 칸 순환 이동한 뒤 저장하고 cnt를 증가시킨다. Dequeue는 array[F]를 제거한 뒤 F=(F+1)%size로 이동하고 cnt를 감소시킨다. cnt==0이면 Empty, cnt==size이면 Full이다. cnt가 없으면 (R+1)%size==F는 Empty와 Full 모두에서 성립할 수 있다. 반면 이 인덱스 규칙에서 F==R은 원소가 하나인 상태를 나타낸다. 마지막 인덱스에서 처음으로 이동하는 식은 (size−1+1)%size=0이다. 모든 삽입·삭제는 Empty/Full 여부를 확인한 후 수행하며, 각 연산의 시간복잡도는 O(1)이다.

9. Linked Queue(큐)

두 pointer F, R (dummy header 없음)

```
F → [a] → [b] → [c] ← R
```

- Empty(Q) ⟺ F = R = NULL
- **Enqueue(x)**: 새 노드 만들고 R->next = 새노드; R = 새노드;
- 빈 큐에 삽입 시 **같은 코드로 처리 안 됨** → special case 필요

Dummy Header 버전

```
F → [dummy] → [a] → [b] → [c] ← R
```

- Empty: F = R = dummy

- Dummy Header Linked Queue: F는 dummy header를 가리키고, R은 마지막 실제 원소를 가리킨다. 빈 큐에서는 F==R==dummy이며 dummy->next==NULL이다. Enqueue는 빈 큐 여부와 관계없이 새 노드의 next=NULL을 설정한 뒤 R->next=새노드, R=새노드를 수행하므로 O(1)이다. Dequeue는 빈 큐인지 먼저 검사하고, F->next의 노드를 제거한다. 제거한 노드가 R과 같다면 마지막 원소가 제거된 것이므로 R=F로 변경한다. 따라서 마지막 원소 삭제 시 rear pointer의 갱신이 필요하다. Dequeue 역시 O(1)이다.
- 단, 메모리 dummy 1개 추가 낭비

10. Queue(큐) Variants

- **Queue**: FIFO 기본
- **Circular queue(환형 큐)**: 환형 배열 기반
- **Priority queue**: 우선순위 기반 (Ch7)
- **Double-ended queue (deque)**: 양끝에서 insert/delete 모두 가능

시간복잡도 정리

구현	Insert	Delete	Search	비고
Array(배열) list	$O(n)$	$O(n)$	$O(n)$	shift 필요
Linked list	$O(1)$*	$O(1)$*	$O(n)$	*삽입·삭제 위치의 선행 노드 포인터가 주어질 때 (위치 탐색 비용 별도)
Cursor(커서) list	$O(1)$	$O(1)$	$O(n)$	
Array(배열) stack	$O(1)$	$O(1)$	—	
Linked stack	$O(1)$	$O(1)$	—	
Circular queue(환형 큐)	$O(1)$	$O(1)$	—	
Linked queue	$O(1)$	$O(1)$	—	